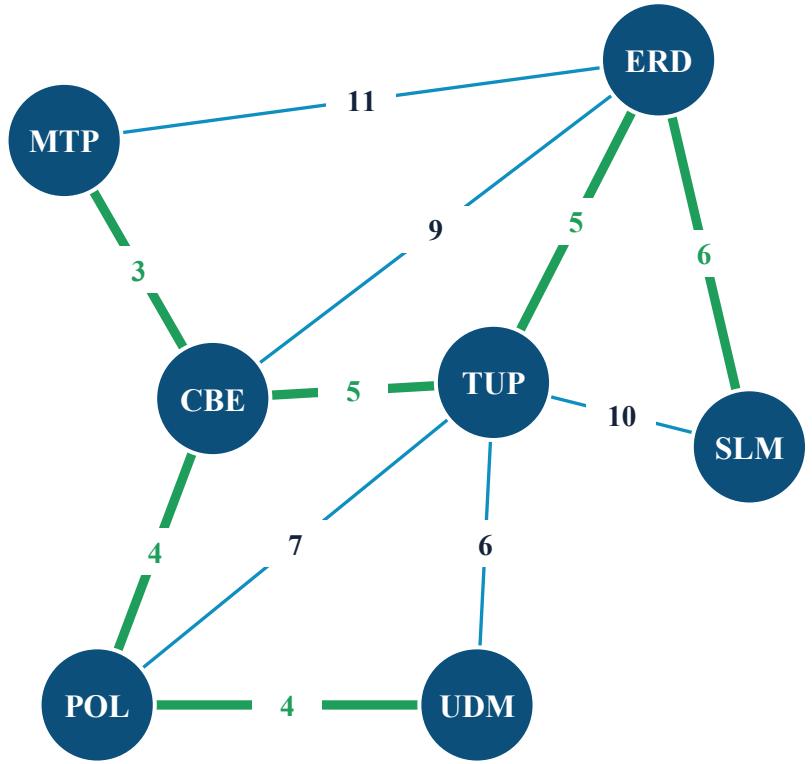
23I501 – DESIGN AND ANALYSIS OF ALGORITHMS
BATCH 2

Telecom: Fibre Rollout Connecting Towns



7 towns 11 feasible fibre routes green = minimum-cost network

PRESENTED BY

24I331 MESHYASHRI S	**24I313** DHARSHINI S	**24I353** SAMYUKTHA S	**24I359** SOWNDHARYASHRI P	**24I364** SUBARIKA N

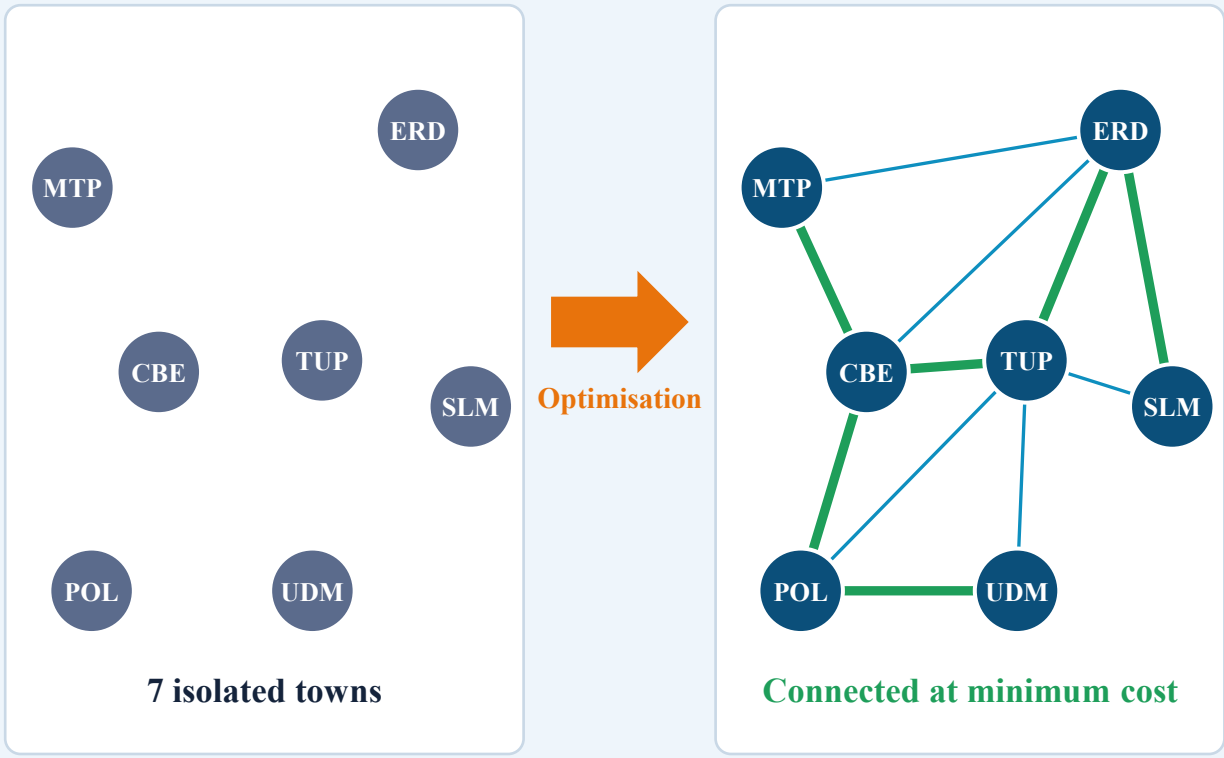
The Challenge – Connecting Every Town

A telecom operator plans to connect every town in a district cluster using optical fibre cables. However:

- Every possible connection has a different trenching cost
- Any town can relay communication to other towns
- Every town must be reachable
- The total installation cost should be minimum

Parameter	Value
Number of towns in example	7
Possible fibre connections	11
Actual network towns	1,200
Actual feasible links	4,000

CBE – Coimbatore , TUP – Tiruppur , ERD – Erode , SLM – Salem , POL – Pollachi , MTP – Mettupalayam , UDM – Udumalpet



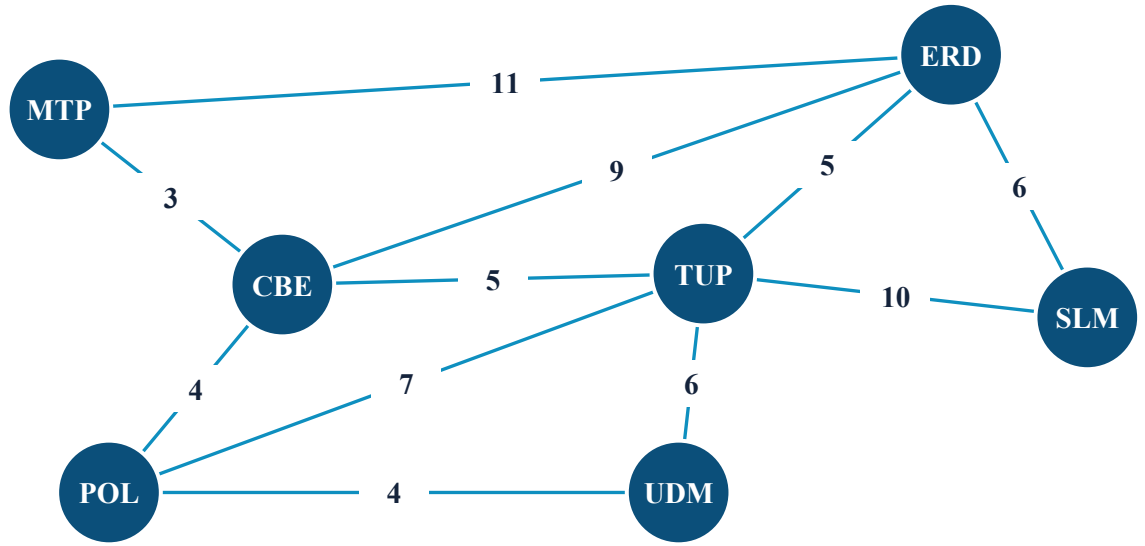
How can we connect all seven towns using the minimum possible trenching cost?

Turning a Telecom Problem into a Graph

G = (V, E) |V| = 7 |E| = 11

Real-world meaning	→	Graph concept
Town	→	Vertex
Feasible fibre link	→	Edge
Trenching cost (₹ lakh)	→	Weight w(u,v)
All towns connected	→	Spanning tree
Minimum-cost network	→	MST

Problem type: OPTIMISATION (minimise cost)



Edge labels = trenching cost in ₹ lakh

INPUT

- Undirected weighted graph G = (V, E)
- 7 towns, 11 links, w = cost in ₹ lakh

OUTPUT

- A set T ⊆ E of fibre links that connects all towns

OBJECTIVE FUNCTION

minimise Σ w(u,v)
over all links (u,v) ∈ T

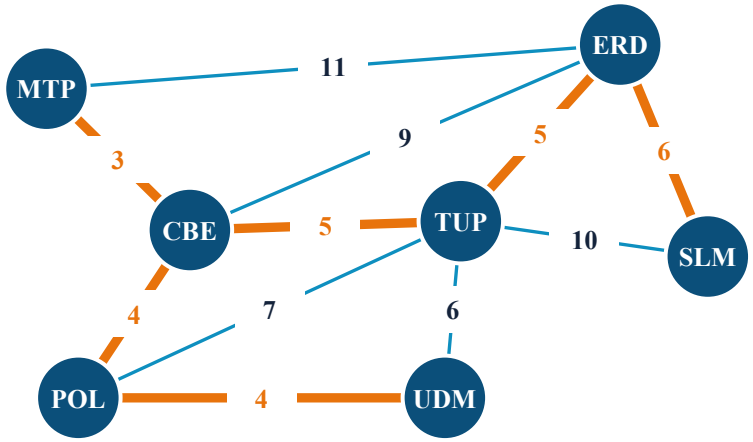
CONSTRAINTS

- All towns connected
- No cycles (no wasted link)
- Exactly |V| − 1 = 6 links

Two Paths to the Minimum-Cost Network

Kruskal's Algorithm

Sort all edges by cost; take the cheapest edge that creates no cycle.

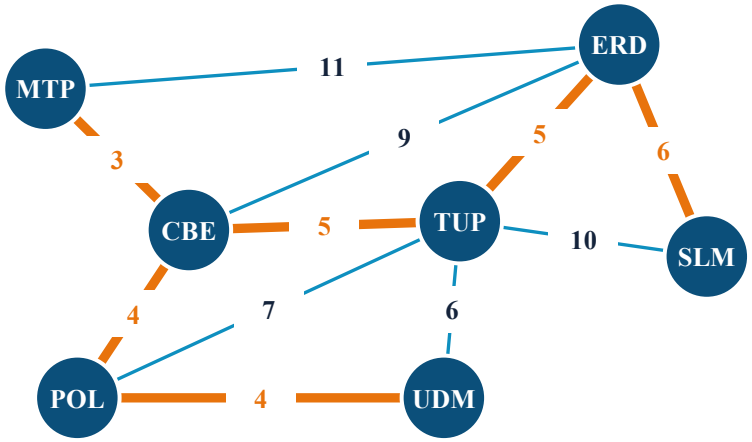


Result	Kruskal	Prim
Selected edges	6	6
Total cost	₹27 lakh	₹27 lakh
All towns connected	Yes	Yes

Both give ₹27 lakh

Prim's Algorithm

Start at CBE; keep adding the cheapest link leaving the tree.

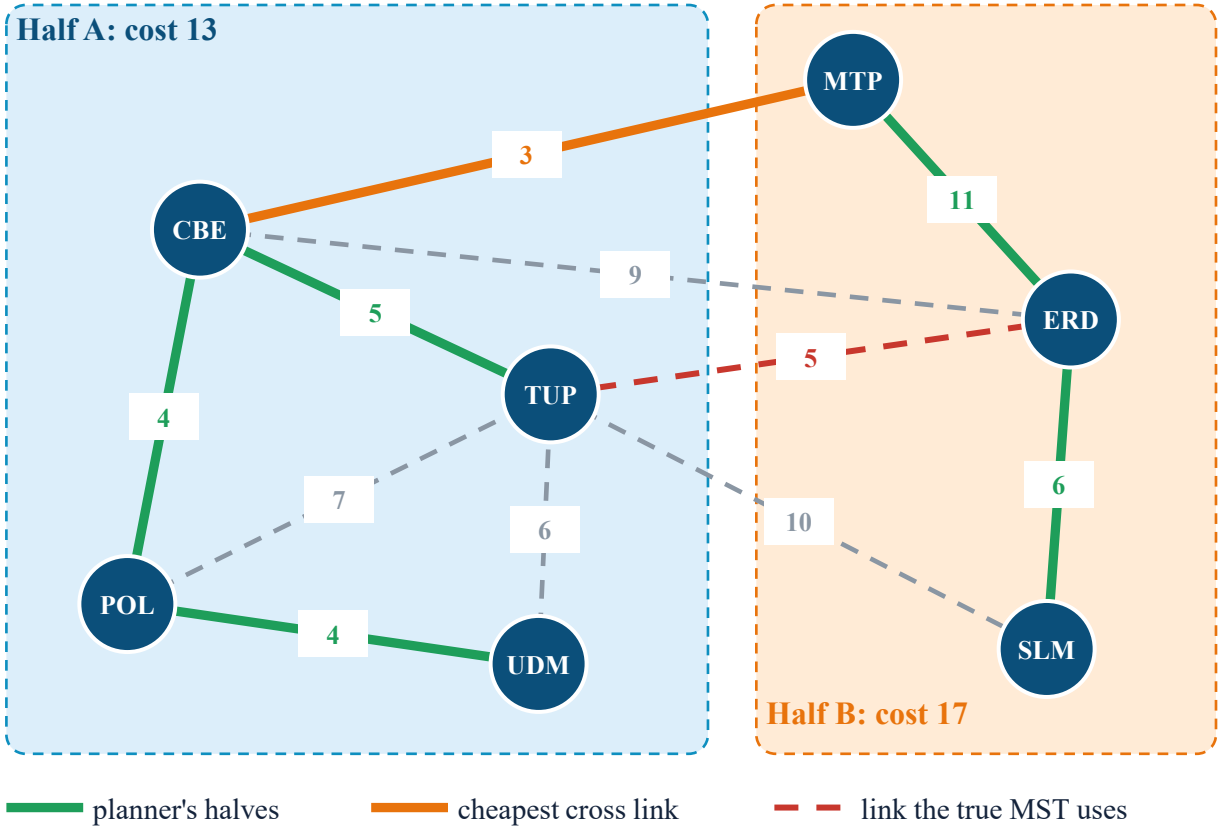


DO THEY PICK THE SAME LINKS? YES – THE SAME SIX (COST IN ₹ LAKH)

- CBE–MTP = 3
- CBE–POL = 4
- POL–UDM = 4
- CBE–TUP = 5
- TUP–ERD = 5
- ERD–SLM = 6

They even pick them in the same order here, and the MST is unique, so every correct algorithm must return it. The intermediate steps (slides 12–13) show where the two strategies differ.

Split into Two Halves: Is It Optimal?



On the instance: NOT optimal

Cost = 13 + 17 + 3 = 33 > 27 (₹6 lakh more).

Inside Half B, MTP can only join through MTP–ERD (11). The true MST instead uses two crossing links, CBE–MTP (3) and TUP–ERD (5): saving 11 − 5 = 6.

In general: disproved

- The counterexample above refutes “always optimal”.
- The result is always a valid spanning tree, so cost ≥ MST; equality holds only if the MST crosses the cut exactly once.
- The cut property makes only the cheapest crossing edge safe. An MST may cross a cut many times.
- A half may even be disconnected, so “solve each half” can be impossible.

Verdict: split-and-join is NOT optimal on this instance and not in general.

Choosing the Representation: Sparse vs Dense

1,200 towns (vertices)

4,000 feasible links (edges)

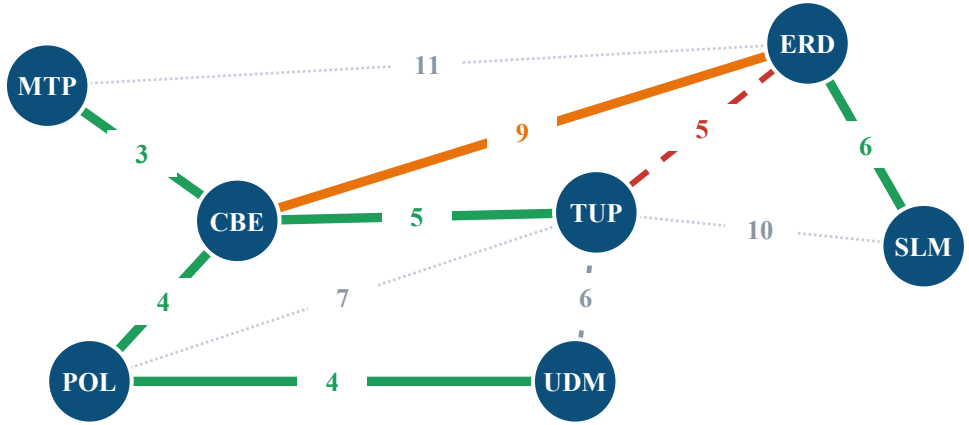
0.56% of the 719,400 possible pairs

	Real network: V = 1,200, E = 4,000	If every pair is feasible: E = 719,400
Density	Sparse (about 3.3 links per town)	Dense (every pair is a link)
Representation	Edge list + DSU (adjacency list + heap also fine)	Adjacency matrix + key[] and visited[] arrays
Algorithm	Kruskal: sort the 4,000 links, scan with DSU	Prim, array version, $O(V^2)$
Approx. operations	$E \log_2 E \approx 48{,}000$ (Prim + heap ≈ 41,000)	$V^2 \approx 1.44$ million (Kruskal ≈ 14 million; Prim + heap ≈ 7.4 million)
Memory	4,000 triples (u, v, w) + 1,200 DSU parents	1.44 million matrix cells (vs 719,400 triples)
Why	Pay only for links that exist; sorting 4,000 items is trivial	Plain array scans beat sorting 719,400 edges; no heap or DSU needed

What changes: the best data structure follows the density. **Sparse: store edges.** **Dense: store a matrix and drop sorting.**

When the Real World Adds Constraints

Case A : Government mandates CBE–ERD

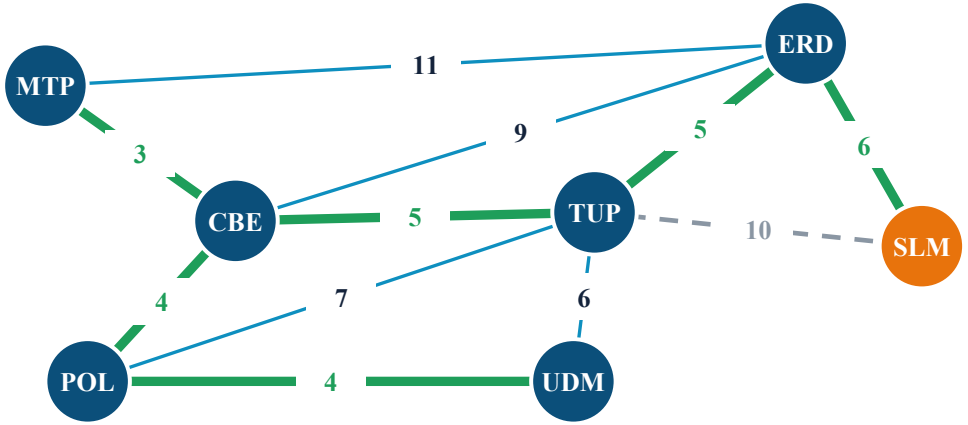


Method: force CBE–ERD first (contract it), then run Kruskal.

Cost: 9 + 3 + 4 + 4 + 5 + 6 = **₹31 lakh** (+₹4). TUP–ERD (5) now closes a cycle and is rejected.

Still optimal: trees containing e correspond one-to-one to spanning trees of G/e plus w(e); Kruskal is optimal on G/e by the cut property.

Case B : Salem may end at most one fibre



Check: the MST already has Salem at degree 1 (only ERD–SLM), so it is feasible and stays optimal: **₹27 lakh**.

General k = 1 method: Salem must be a leaf: MST of the other six towns (21) + cheapest Salem link (6) = 27.

Complexity class: **P** – a bound on one vertex is polynomial (matroid intersection, Gabow–Tarjan 1984). Bounds on every vertex (degree ≤ 2) are NP-hard, where greedy fails.

Choosing the Right Data Structure

Operation	Edge list (chosen)	Adjacency list	Adjacency matrix
Space	O(E)	O(V + E)	$O(V^2)$
Edge lookup (u, v)	O(E)	O(deg)	O(1)
List all edges	O(E)	O(V + E)	$O(V^2)$
Neighbours of v	O(E)	O(deg)	O(V)
Sort edges by weight	O(E log E), in place	O(E log E) after extraction	O(E log E) after $O(V^2)$ scan
Fit for Kruskal	Best: flat list of (u, v, w)	Needs extraction first	Wasteful when sparse

DSU VARIANTS (CYCLE DETECTION)

Variant	Find	Union	Space
Component-id array	O(1)	O(V)	O(V)
Trees, no heuristics	O(V) worst	O(V) worst	O(V)
Path compression + union by rank (chosen)	**$O(\alpha(V))$ amortised**	**$O(\alpha(V))$ amortised**	**O(V)**

Disjoint Set Union

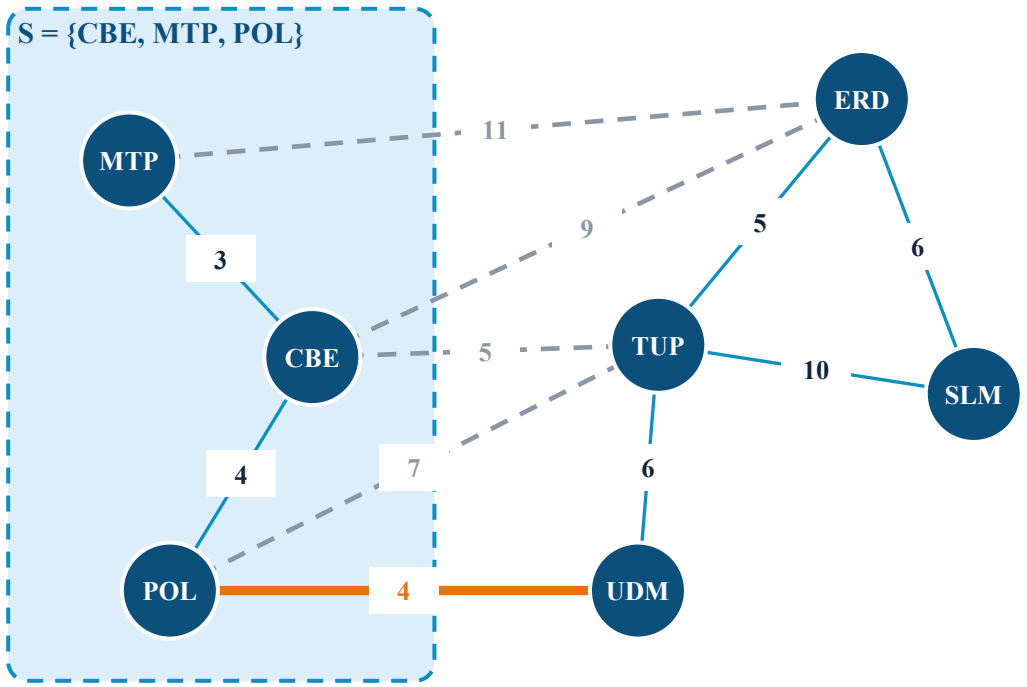
- MAKESET(v): every town starts alone
- FIND(v): root of v's component
- UNION(a, b): merge two components
- Compression flattens paths; rank keeps trees shallow
- FIND(u) = FIND(v) means a cycle

```
edges = [("CBE","MTP",3),
("CBE","POL",4),
("POL","UDM",4), ...]
```

Edge list: each link stored once as (u, v, w).

Why edge list + DSU? V = 1,200 and E = 4,000 is sparse: store only existing links, sort once, and test cycles in near-constant time.

Why Greedy Works for MST



Cut (S, V − S): five links cross it (4, 5, 7, 9, 11). The cheapest, POL–UDM = 4, is in the MST.

SELECTED TECHNIQUE : GREEDY

Greedy-choice (cut) property. Let F be contained in some MST T, and let (S, V − S) be a cut that no edge of F crosses. If e is a minimum-weight edge crossing the cut, then F ∪ {e} is contained in some MST.

Proof. If e ∈ T, done. Otherwise T + e contains a cycle, which crosses the cut at another edge f ∈ T with w(e) ≤ w(f). Then T′ = T − f + e is a spanning tree with w(T′) ≤ w(T), so T′ is an MST containing F ∪ {e}. (QED)

Optimal substructure. After choosing a safe edge e, the rest of an MST is an MST of the contracted graph G/e.

Kruskal's rule: the cheapest edge joining two different components is always safe.

Greedy works because a locally cheapest safe edge can always be extended to a globally optimal tree.

Why Not the Other Four Techniques?

Technique	Why it is unsuitable	Concrete evidence
Divide and Conquer	Halves are not independent: the optimal tree may cross the split several times, so “solve halves + one cross link” is not a valid combine step.	Split {CBE, TUP, POL, UDM} | {MTP, ERD, SLM}: 13 + 17 + 3 = 33 ≠ 27. The MST crosses twice (CBE–MTP, TUP–ERD).
Dynamic Programming	MST has optimal substructure, but a table over vertex subsets is far costlier than greedy, which needs no table at all.	7 towns: $2^7 = 128$ subsets, $3^7 = 2{,}187$ split checks. 1,200 towns: $2^{1200} \approx 10^{361}$ states.
Backtracking	Include/exclude choice per link; even with cycle pruning the search space stays exponential.	11 links: $2^{11} = 2{,}048$ subsets (462 six-edge sets). 4,000 links: C(4000, 1199) > 10^{1000}. Measured: V = 8 needs 1,184,040 sets, 437 ms vs about 4 µs for Kruskal.
Branch and Bound	Needs a bound that is cheap and tight. The cheap bound (cheapest remaining edges, ignoring connectivity) is weak; a tight bound is the MST itself (circular).	Two 4-town clusters of weight-1 links joined by one weight-100 link: bound = 7, optimum = 106, so almost no partial tree is pruned.

Takeaway: greedy is preferred because the cut property gives a safe edge at every step, with no search and no table.



How Kruskal Builds the Network

PSEUDOCODE

```
KRUSKAL(V, E):
   F ← ∅ ;  total ← 0
   sort E by weight, ascending
   for each v in V: MAKESET(v)
   for each edge (u, v, w) in E:
      if FIND(u) ≠ FIND(v):
         F ← F ∪ {(u, v)}
         UNION(u, v)
         total ← total + w
         if |F| = V − 1: break
   return F, total
```

FIND(x): while parent[x] ≠ x:
parent[x] ← parent[parent[x]]; x ← parent[x]
UNION(a, b): attach lower rank under higher rank

State definition

F = accepted links (a forest); the DSU partition = connected components of (V, F).

Invariant

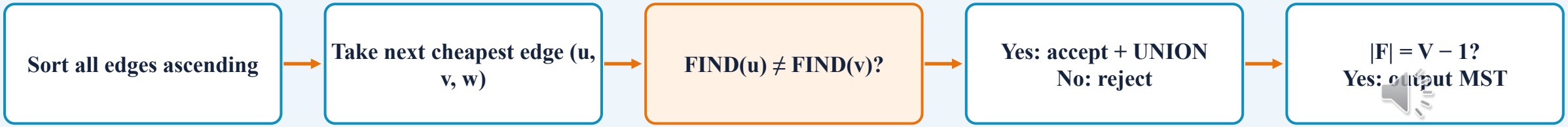
F is contained in some MST.

Greedy rule

Take the cheapest unexamined edge that joins two different components; skip any edge inside one component.

Stop

After |V| − 1 = 6 accepted links.

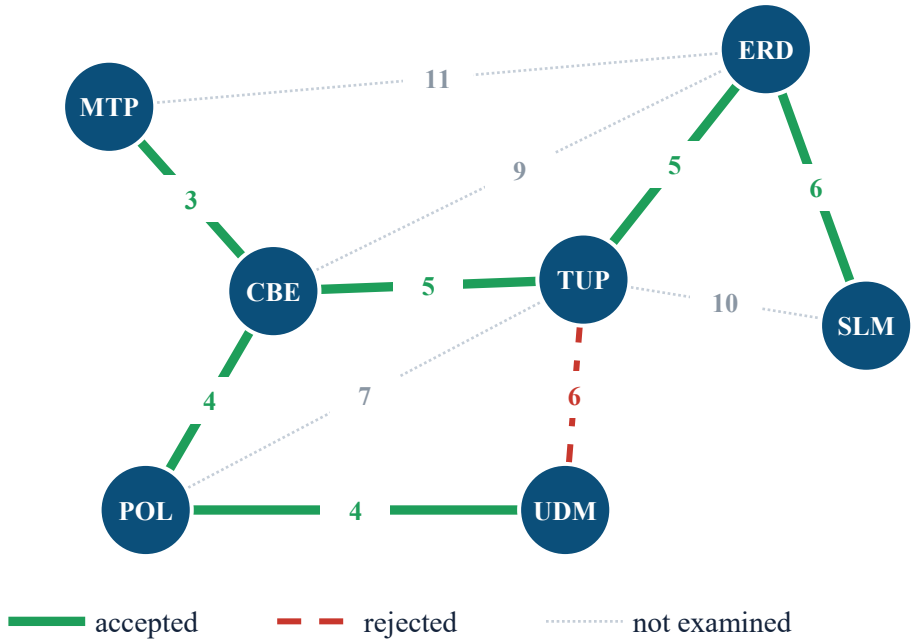


Following Every Connection: Kruskal Trace

#	Edge	Cost	Decision	Components left	Total
1	CBE–MTP	3	Accept	6	3
2	CBE–POL	4	Accept	5	7
3	POL–UDM	4	Accept	4	11
4	CBE–TUP	5	Accept	3	16
5	TUP–ERD	5	Accept	2	21
6	TUP–UDM	6	Reject (cycle)	2	21
7	ERD–SLM	6	Accept	1	27
8	POL–TUP	7	not examined	–	–
9	CBE–ERD	9	not examined	–	–
10	TUP–SLM	10	not examined	–	–
11	MTP–ERD	11	not examined	–	–

Start: 7 singleton components. Sorted costs: 3, 4, 4, 5, 5, 6, 6, 7, 9, 10, 11.

TUP–UDM is rejected : FIND(TUP) = FIND(UDM), since both are already linked through CBE–POL–UDM. After 6 accepts the loop stops.



3 + 4 + 4 + 5 + 5 + 6 = 27

Total = ₹27 lakh

Growing the Network from Coimbatore: Prim Trace

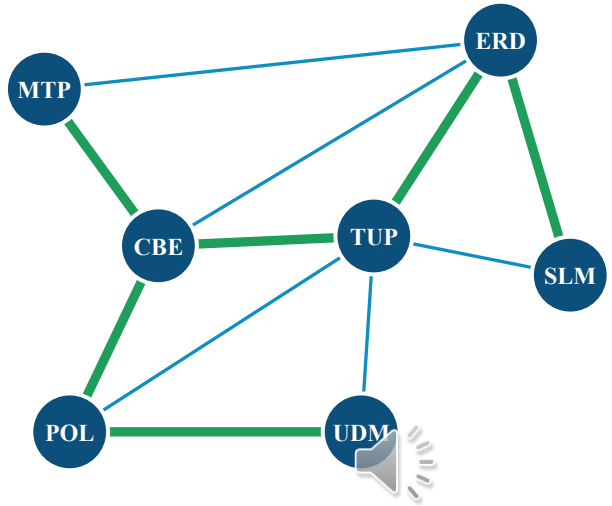
Step	Tree so far	Picked (cheapest frontier)	Other candidates	Total
1	{CBE}	CBE–MTP 3	CBE–POL 4, CBE–TUP 5, CBE–ERD 9	3
2	+ MTP	CBE–POL 4	CBE–TUP 5, CBE–ERD 9, MTP–ERD 11	7
3	+ POL	POL–UDM 4	CBE–TUP 5, POL–TUP 7, CBE–ERD 9, MTP–ERD 11	11
4	+ UDM	CBE–TUP 5	UDM–TUP 6, POL–TUP 7, CBE–ERD 9, MTP–ERD 11	16
5	+ TUP	TUP–ERD 5	CBE–ERD 9, TUP–SLM 10, MTP–ERD 11	21
6	+ ERD	ERD–SLM 6	TUP–SLM 10	27

Feature	Kruskal	Prim
Works on	Edges	Growing tree
Start	None needed	CBE
Cycle check	DSU	visited[]
Needs	Sorted edges	Min-heap
Output	₹27 lakh	₹27 lakh

Same links? Yes. The same six links, even in the same order here: every cheapest safe link happened to touch the tree growing from CBE.

Intermediate difference: Kruskal examined TUP–UDM (6) and rejected it through DSU; Prim never picked it, because once TUP joined the tree it was no longer a frontier link.

They differ in general: if ERD–SLM cost 1, Kruskal would accept it first (a second component), Prim only at the end. The final tree is the same.



Why Can We Trust Kruskal?

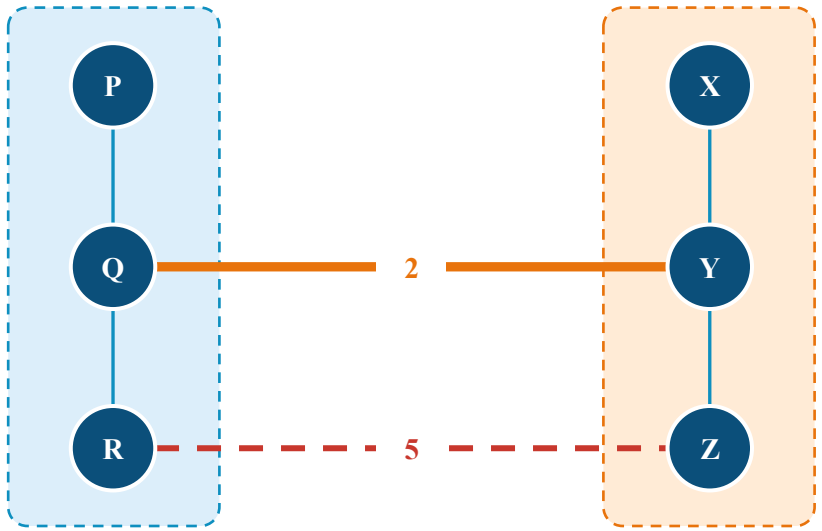
Loop invariant: after every iteration, the accepted set F is contained in some MST.

1 Initialisation. F = ∅ is contained in every MST.

2 Maintenance, edge e = (u, v), cheapest unexamined.

- Reject case: u and v are in one component, so e would close a cycle; no MST containing F contains e, and F is unchanged.
- Accept case: u and v lie in different components. Take the cut (component of u, rest). No edge of F crosses it, and e is the cheapest edge crossing it, because every cheaper edge was already examined and lies inside a component. By the cut property F ∪ {e} is contained in some MST.

3 Termination. After |V| − 1 accepts, F is connected, acyclic and contained in an MST, so F is an MST. (Fewer accepts would mean G is disconnected.)



Cut: e = 2 (orange) is safe; an optimal tree using f = 5 (red) swaps f → e.

$$w(T') = w(T) - (5 - 2) \leq w(T)$$

Kruskal(G) = MST(G)

Can This Algorithm Handle 1,200 Towns?

KRUSKAL (EDGE LIST + DSU)

Step	Cost
Sort E edges	$\Theta(E \log E)$
MAKESET for V towns	$O(V)$
$\leq$ E FIND/UNION pairs	$O(E\ \alpha(V))$
Time (total)	**$O(E \log E) = O(E \log V)$**
Space	**$O(V + E)$**

$E \leq V^2$, so $\log E \leq 2 \log V$. $\alpha(V) \leq 4$ for any practical V, so sorting dominates.

PRIM

Version	Time	Space
Adjacency list + binary heap	$O(E \log V)$	$O(V + E)$
Adjacency matrix + array	$O(V^2)$	$O(V^2)$

≈ 48,000

Kruskal steps, E = 4,000

≈ 41,000

Prim + heap steps

BEST, WORST AND PRACTICAL CASES (KRUSKAL)

Case	Time	When
Best	$O(E)$ if pre-sorted; $\Theta(E \log E)$ otherwise	Edges already sorted and the first $V - 1$ are accepted
Worst	$\Theta(E \log E)$	Sorting dominates; the loop may scan every edge
Practical	≈ 48,000 steps	Sparse network: about 3.3 links per town, early stop after 6 … 1,199 accepts

If every pair is feasible (E = 719,400): Kruskal ≈ 14 million steps vs matrix Prim $O(V^2)$ ≈ 1.44 million, so matrix Prim wins.

Implementation: Kruskal vs Brute Force

KRUSKAL WITH DSU

```
def find(p, x):
    while p[x] != x:
        p[x] = p[p[x]]; x = p[x]
    return x

def kruskal(V, edges):
    p, rank = list(range(V)), [0] * V
    mst, total = [], 0
    for u, v, w in sorted(edges, key=lambda e: e[2]):
        ru, rv = find(p, u), find(p, v)
        if ru != rv:
            if rank[ru] < rank[rv]: ru, rv = rv, ru
            p[rv] = ru
            rank[ru] += rank[ru] == rank[rv]
            mst.append((u, v, w)); total += w
            if len(mst) == V - 1: break
    return mst, total
```

REJECTED ALTERNATIVE: BRUTE FORCE

```
from itertools import combinations

def brute(V, edges):
    best = None
    for combo in combinations(edges, V - 1):
        if spans(V, combo):  # V-1 edges, connected
            c = sum(w for _, _, w in combo)
            if best is None or c < best: best = c
    return best
```

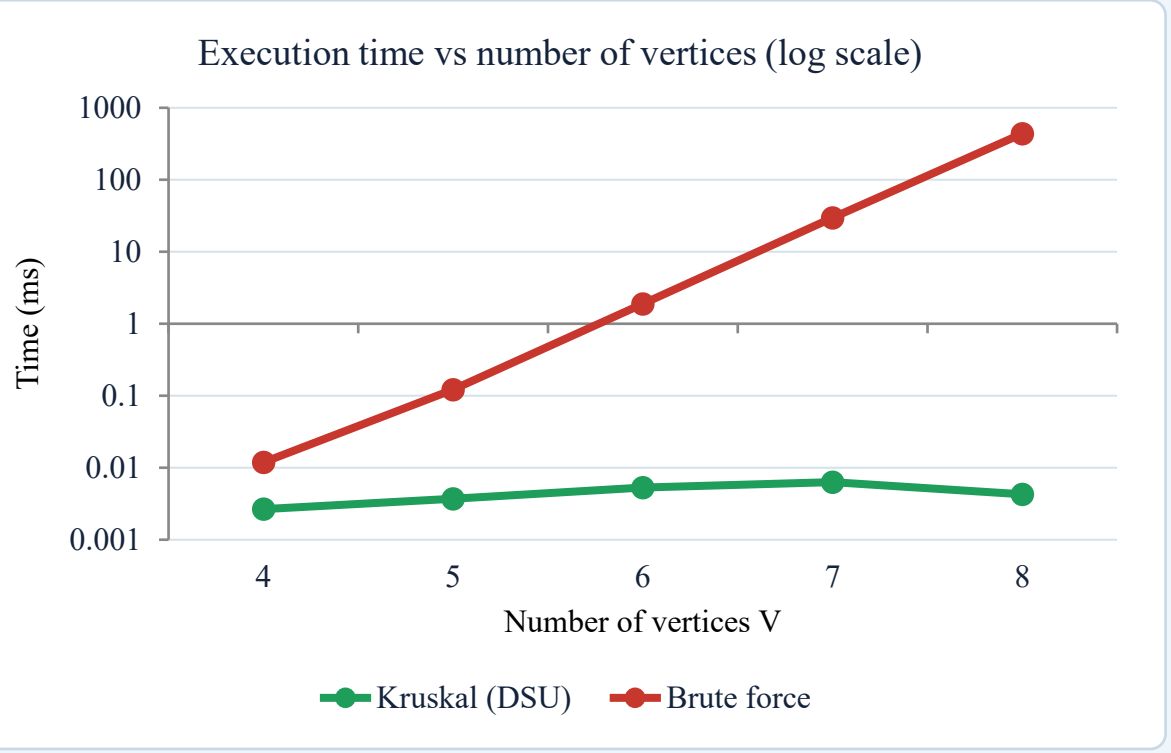
```
t0 = time.perf_counter()
kruskal(V, edges)
elapsed = time.perf_counter() - t0
```

Complete weighted graphs, random weights 1–50, V = 4 … 8; median of repeated runs.

Brute force tests every one of C(E, V − 1) subsets; Kruskal sorts once and scans. Full script: kruskal_experiment.py



Theory vs Actual Execution



V	E	Subsets C(E, V−1)	Kruskal (µs)	Brute (ms)	µs per subset
4	6	20	2.7	0.012	0.59
5	10	210	3.7	0.121	0.58
6	15	3,003	5.3	1.893	0.63
7	21	54,264	6.3	29.756	0.55
8	28	1,184,040	4.3	436.569	0.37

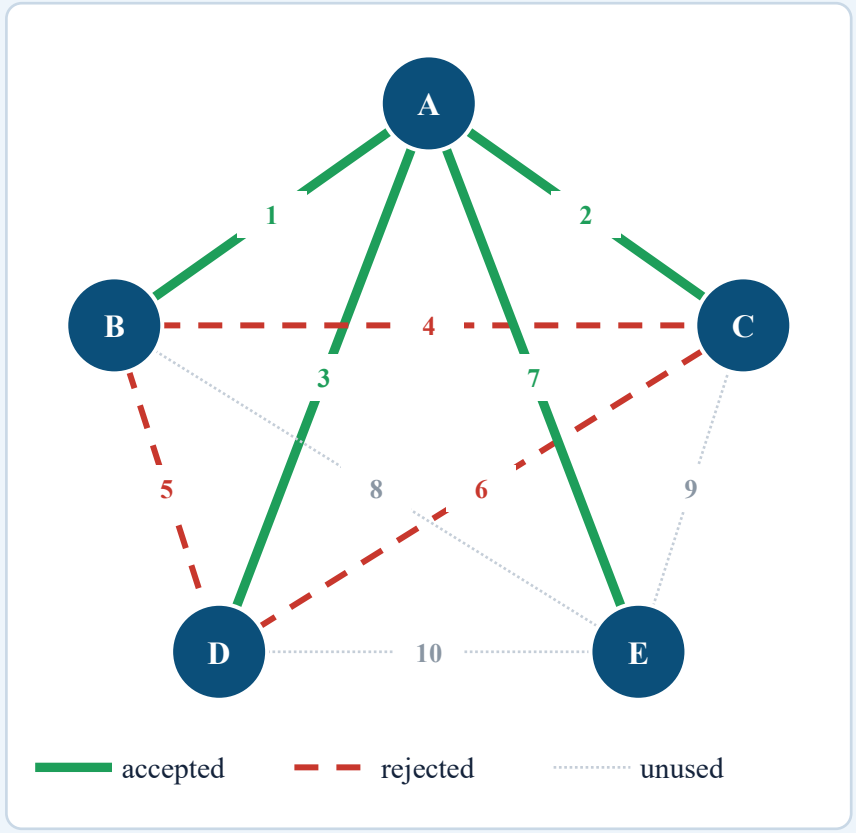
Derived: brute force $\Theta(C(E, V-1)\cdot V)$, Kruskal $O(E \log E)$ with $E = V(V-1)/2$.

At V = 8: brute force ≈ 437 ms vs Kruskal ≈ 4 µs, a gap of about 100,000×.

- Brute-force time tracks the subset count C(E, V−1) at roughly 0.4–0.6 µs per subset, so it grows exponentially, as derived.
- Kruskal's predicted E log E growth (about 16 → 135 steps) is smaller than timer noise, so V = 8 reads lower than V = 7.
- Five small sizes show the trend; this is not a large-scale telecom benchmark.

When Does Kruskal Do the Most Work?

Edge	Weight	Verdict
A–B	1	Accept
A–C	2	Accept
A–D	3	Accept
B–C	4	Reject
B–D	5	Reject
C–D	6	Reject
A–E	7	Accept
B–E	8	not needed
C–E	9	not needed
D–E	10	not needed



Behaviour

- First three edges accepted: A, B, C, D connected
- Next three rejected: each closes a cycle
- A–E accepted: E joins; MST done after 7 of 10 edges

Complexity

$E = V(V - 1)/2$

$O(E \log E) = O(V^2 \log V)$

Generalised: K_V with the new vertex attached only by the heaviest edges makes Kruskal examine C(V−1, 2) + 1 of C(V, 2) edges, a fraction that tends to 1.

Worst case: the dense complete graph ($E = V^2/2$) forces the largest sort and many cycle rejections before the last town is reached.

From an Algorithm to a Real Library

NetworkX – Python graph library

```
import networkx as nx
T = nx.minimum_spanning_tree(G, algorithm="kruskal")
```

Built-in Kruskal (default), Prim and Borůvka. Official documentation: networkx.org

Published papers

• J. B. Kruskal (1956), On the shortest spanning subtree of a graph and the traveling salesman problem, Proc. AMS 7(1).

• R. C. Prim (1957), Shortest connection networks and some generalizations, Bell System Technical Journal 36(6).

HOW OUR VERSION DIFFERS

Aspect	Our implementation	NetworkX
Graph storage	Edge list of (u, v, w) tuples	Dict-of-dicts adjacency
Cycle detection	Hand-written DSU (rank + compression)	Built-in union-find helper
Algorithms	Kruskal; Prim traced by hand	Kruskal, Prim, Borůvka
Input	One connected 7-town instance	Any graph; returns a spanning forest if disconnected
Constraints	Forced link and degree rule handled by hand	Not built in
Purpose	Understanding, tracing, timing	Production-grade library



Conclusion and Requirement Coverage

₹27 lakh

minimum cost for the original seven-town network

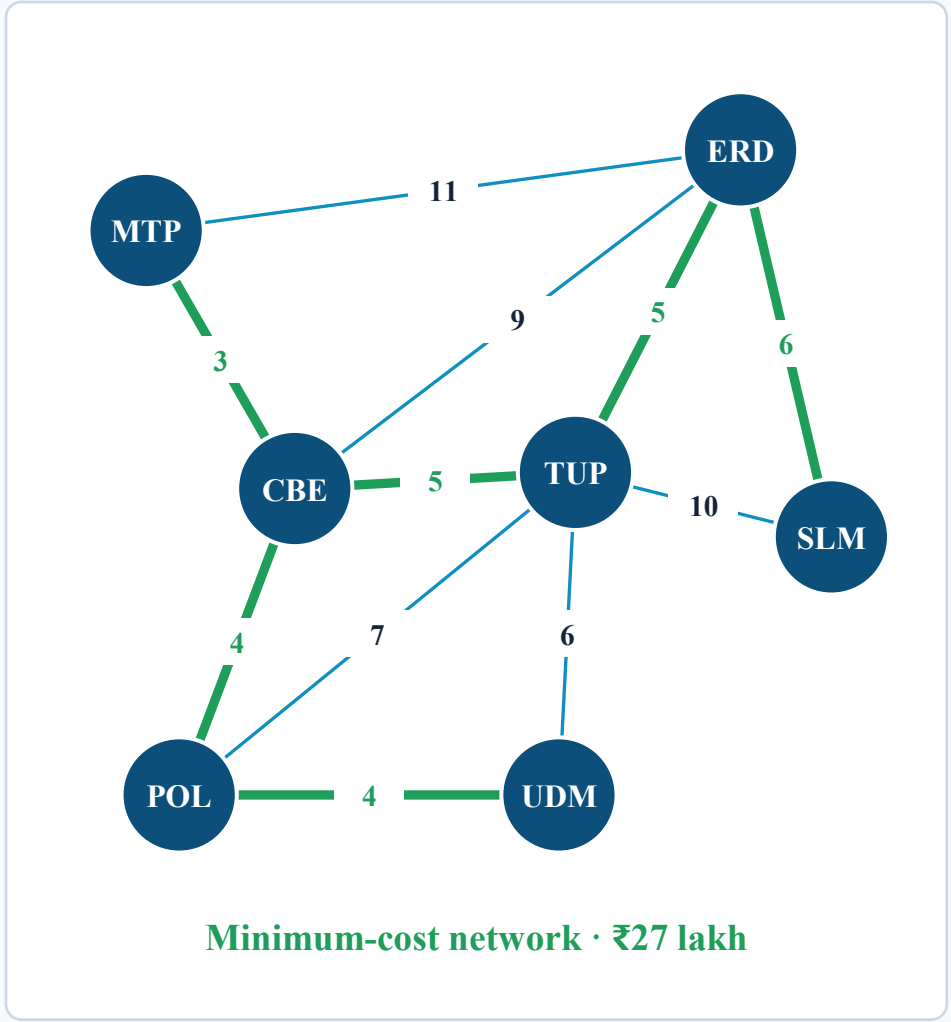
"By using Minimum Spanning Tree algorithms, we can connect all towns without unnecessary fibre links while minimising the installation cost."

- Kruskal and Prim agree: 27
- Split-and-join: 33 (not optimal)
- CBE–ERD forced: 31
- Salem degree ≤ 1: 27 (in P)

Five questions	Slides
Q1 Graph model + proof of optimality	3, 9, 14
Q2 Two algorithms compared	4, 12, 13
Q3 Split into halves	5
Q4 Representation, 1,200 towns	6
Q5 Forced link, Salem degree	7

Components and extras	Slides
1 Problem modelling	3
2 Data structure analysis	6, 8
3 Design technique + proof of property	9
4 Rejection of four techniques	10
5 Algorithm (pseudocode)	11
6 Trace (Kruskal and Prim)	12, 13
7 Complexity analysis	15
8 Changed requirement	7
Proof of correctness	14
Implementation + empirical analysis	16, 17
Worst-case input	18
Real-world reference	19

THANK YOU



Connecting Towns. Minimising Cost.